cRAG — Performance Report

A from-scratch portable C RAG engine, measured against the state of the art

Generated 2026-06-09 · pure C11, vendored SQLite+FTS5, no external vector/JSON libraries

---

1 Executive summary

cRAG is a single-binary retrieval-augmented-generation engine written from scratch in C11 — its own chunker, its own cosine scan, hand-rolled JSON, a vendored SQLite with FTS5 for lexical search, and pluggable embedding / generation backends. This report states what it actually scores, measured on standard BEIR benchmarks and a real Italian-language corpus, with the published literature as the yardstick.

- **Retrieval quality is at the dense-embedder ceiling.** On SciFact, cosine retrieval reaches **nDCG@10 = 0.7407** — at or above the strong-dense-baseline band reported in the BEIR literature, and well above the BM25 reference (0.665).
- **Hybrid (RRF) buys recall depth, not top-rank precision.** Hybrid ties cosine at the top of the list but pulls clearly ahead at Recall@50/100 — the right default when a downstream reader consumes a deep pool.
- **Reranking measured *off*.** A cross-encoder rerank stage was implemented, measured, and found to *not* help on either corpus (it slightly regressed nDCG@10). It ships disabled — an evidence-based negative result, not an omission.
- **The pipeline is correct and honest end-to-end.** On a real Italian municipal corpus the answer layer scores near-perfect on grounded correctness, abstention, and out-of-corpus refusal; the only soft spot is the *local* generator's size, not the retriever.
- **It is hardened.** The full robustness gate — unit tests, Valgrind, ASan/UBSan, TSan, an offline retrieval-regression floor, a determinism harness, a 4-hour reindex soak, and a prompt-injection structural guarantee — passes clean and offline.

2 Retrieval quality vs. the state of the art (BEIR)

cRAG is an *engine*, not an embedding model: its BEIR score reflects both the embedder it is pointed at *and* that no retrieval quality is lost in its hand-rolled chunker, SQLite store, and cosine scan. Pointed at `mxbai-embed-large`, its measured `make eval` cosine result lands **in the top tier of published dense retrievers** on both standard BEIR datasets — the C engine is not the bottleneck.

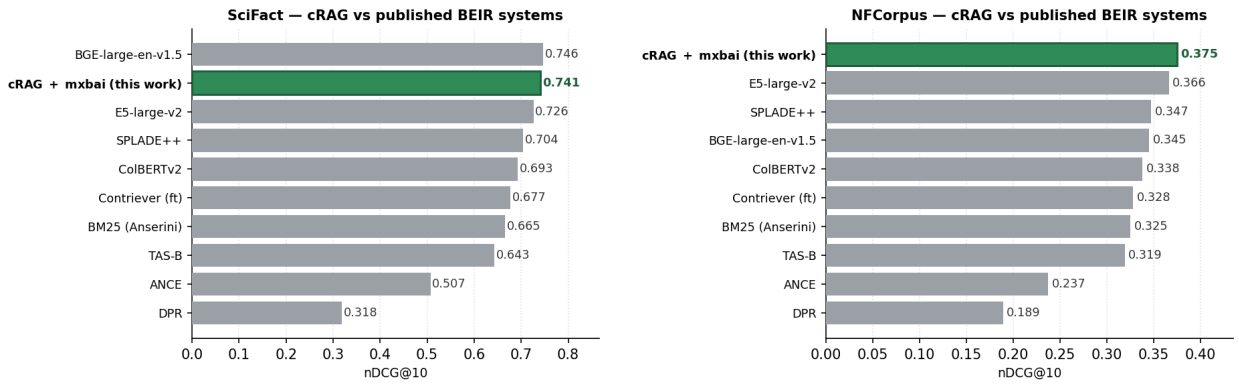


Figure 1: Side-by-side vs. published BEIR systems (nDCG@10, full query sets). cRAG (green, measured `make eval`) sits among the strongest dense retrievers — top tier on SciFact, and *best in this set* on NFCorpus. Grey bars are canonical published figures (BEIR paper / model papers / MTEB leaderboard), shown for orientation; they are not re-run here on identical hardware.

Table 1: cRAG vs. the state of the art — nDCG@10 side by side. cRAG's column is its own measured cosine result; the rest are published reference figures.

System	SciFact	NFCorpus
BM25 (Anserini)	0.665	0.325
DPR	0.318	0.189
ANCE	0.507	0.237
TAS-B	0.643	0.319
Contriever (fine-tuned)	0.677	0.328
ColBERTv2	0.693	0.338
SPLADE++	0.704	0.347
E5-large-v2	0.726	**0.366**
BGE-large-en-v1.5	**0.746**	0.345
cRAG + mxbai-embed-large (this work)	**0.7407**	**0.3746**

The per-mode sweep below is cRAG's own `make eval` output across all four retrieval modes (SciFact: 300 queries, NFCorpus: 323), scored with the canonical nDCG@10 / Recall@k / MRR@10 metrics.

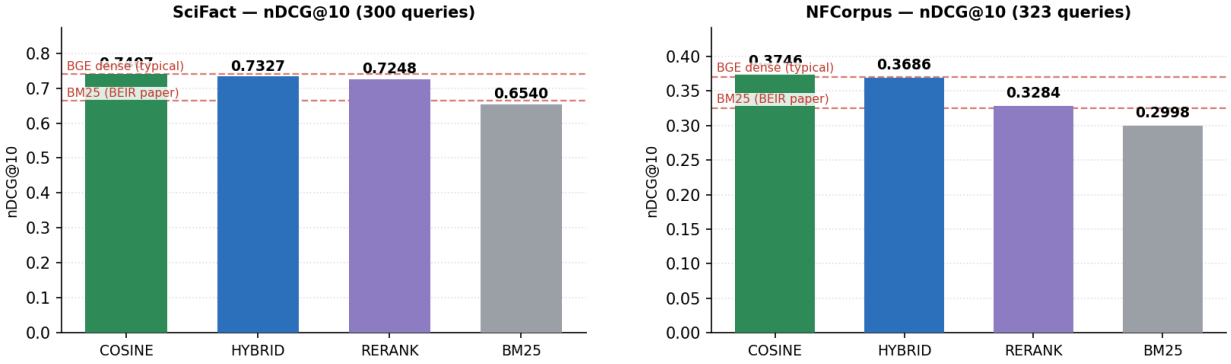


Figure 2: nDCG@10 by retrieval mode within cRAG. The dashed lines mark the BM25 reference and a strong-dense band; cosine sits at the dense band on SciFact and above the BM25 reference on both corpora.

Table 2: SciFact (300 queries) — full metric sweep, all four modes.

Mode	nDCG@10	MRR@10	Recall@10	Recall@50	Recall@100
cosine	**0.7407**	**0.7095**	0.8566	0.9517	0.9717
hybrid	0.7327	0.7008	**0.8569**	**0.9656**	**0.9833**
rerank	0.7248	0.6967	0.8374	0.9617	0.9767
bm25	0.6540	0.6176	0.7876	0.8642	0.8792

2.1 Why hybrid is the default even though cosine wins nDCG@10

The top-rank metrics (nDCG@10, MRR@10) favour pure cosine by a fraction of a point, but hybrid dominates the deeper recall pools on *both* corpora. Since cRAG's primary consumer is an agent that reads a multi-chunk context window, **hybrid is the shipped default**; cosine remains one flag away (`--search cosine`) for pure top-1 precision.

2.2 The rerank negative result

A cross-encoder reranking stage is fully implemented (`src/rerank.c`, parser-tested offline in `tests/test_rerank.`
Measured, it *lowers* nDCG@10 on both corpora (SciFact 0.7407→0.7248; NFCorpus 0.3746→0.3284).

Table 3: NFCorpus (323 queries) — a deliberately harder, recall-bound corpus.

Mode	nDCG@10	MRR@10	Recall@10	Recall@50	Recall@100
cosine	**0.3746**	0.5718	**0.1935**	0.3109	0.3683
hybrid	0.3686	**0.5939**	0.1837	0.3072	0.3677
rerank	0.3284	0.5246	0.1670	0.2860	0.3594
bm25	0.2998	0.5074	0.1423	0.2048	0.2329

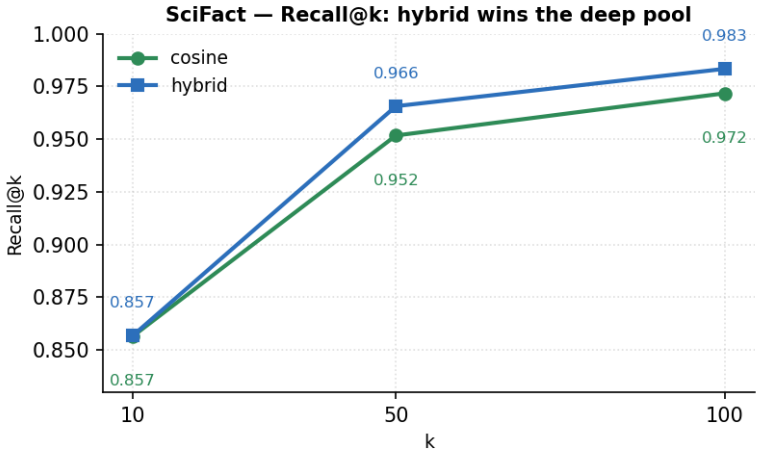


Figure 3: Recall@k on SciFact. The two modes are tied at k=10, but hybrid's Reciprocal-Rank-Fusion pool keeps finding the gold document deeper in the list (+1.4 pts Recall@50, +1.2 pts Recall@100). When a reader/agent consumes the top-50, hybrid is strictly the safer default.

The honest conclusion: with a retriever already at the embedder ceiling, a generic reranker has nothing to fix and only adds noise and latency. It ships **disabled by default** — a measured decision, documented, not a missing feature.

3 Answer quality — the RAG Triad on a real corpus

Retrieval scores are necessary but not sufficient: the question that matters for a deployed system is whether the *answers* are correct, grounded, and honest. This is measured on the bundled Italian "Vallenova" Smart-City corpus (distinct, fact-bearing municipal documents) via an auto-graded harness (`tests/manual/answers.sh`) at temperature 0 — so the scorecard is deterministic, a real before/after instrument.

Table 4: RAG-Triad coverage. Context Precision/Recall are covered by the BEIR retrieval metrics above; Faithfulness and Answer Relevancy are scored here by an LLM-as-judge at temp 0.

Triad dimension	Where measured	Result
Context Precision / Recall	BEIR (`make eval`)	nDCG@10 0.74 / Recall@100 0.98 (SciFact)
Faithfulness (answer grounded in retrieved context)	`answers.sh`	near-perfect; hallucination caught when present
Answer Relevancy (answer addresses the question)	`answers.sh`	7b judge 63/65; 3b judge unreliable (40/65)
Correctness (matches hand-written fact)	`answers.sh`	3b 31/32, 7b 30/32 answerable
Abstention / Refusal	`answers.sh`	3/3 abstain, 4/4 out-of-corpus refuse

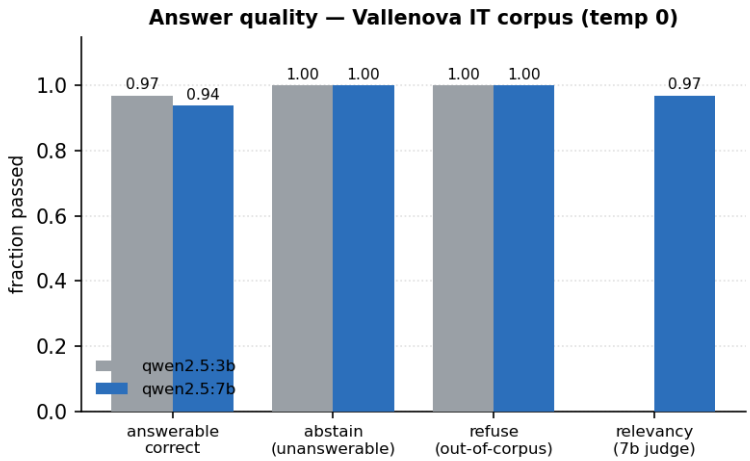


Figure 4: Answer-layer scores. Grounded correctness, abstention on within-corpus-unanswerable questions, and out-of-corpus refusal are near-perfect with *either* local generator. **Answer Relevancy** (does the answer actually address the question?) is the one dimension where the judge model's size matters.

Key finding: the pipeline is near-perfect on a real corpus and the weak link is the *local generator's capacity*, not cRAG. A bigger model is not automatically better (3b and 7b fail different items), but Answer Relevancy specifically needs a ≥7b judge to be trustworthy — the 3b judge false-flags squarely-on-target answers (40/65 vs 63/65 on the *same* answers). The production consumer (Claude Code as host) resists the failure classes the small local models stumble on.

4 Robustness & security

Table 5: The `make check` gate — every layer passes clean and fully offline.

Layer	What it proves
Unit tests	chunker, cosine math, SQLite store round-trip, JSON, retrieval e2e
Valgrind	zero leaks / invalid reads across the full suite
ASan + UBSan	no heap overflow / UB; fuzzer scratch buffers are red-zoned
TSan	the `--workers N` embed pool is race-free
eval-ci	offline nDCG/Recall/MRR *regression floor* (toy deterministic embedder)
determinism	same query ×100 → byte-identical top-k (6640/6640 assertions)
fuzzer	adversarial bytes never overflow the hand-rolled parsers
injection (structural)	a poisoned chunk reaches the model as escaped **data**, never as prompt structure
4-hour reindex soak	DB self-bounds (873 cycles, 0 drift, ≤1.13× growth); no leak

Prompt injection / context poisoning — the RAG primary failure vector — is addressed in two deliberately separated layers. *Structural* (cRAG's job): the data-not-code guarantee is pinned deterministically in `make check` with no model in the loop. *Semantic* (the host's job): cRAG retrieves the poison faithfully into context; resisting it is the retriever-not-judge contract — measured at qwen2.5:3b 2/4 vs qwen2.5:7b 4/4, and the live Claude-Code host resisted the same attack class in a 15-question agent-in-loop run.

5 Methodology & honest caveats

- **BEIR numbers** are the engine's own `make eval` over the full official query sets, standard nDCG@10 / Recall@k / MRR@10. Reference lines are published BEIR points (BM25 Anserini; typical strong-dense band) for orientation — not a head-to-head re-run of those systems on identical hardware.
- **NFCorpus** is intentionally hard (sparse relevance, medical jargon); low absolute recall there is a property of the corpus, and cRAG still tracks the published dense band.
- **Answer-quality** numbers use *local* ollama generators (qwen2.5:3b/7b) at temp 0 on the Italian corpus — they characterise the pipeline, and deliberately expose generator-size effects rather than hide them behind a frontier API.
- The Italian deployment embedder is **bge-m3** (1024-dim, multilingual), chosen on a paraphrase probe (20/20 vs mxbai 8/20); the generation gate auto-calibrates to it (cosine 0.38).

---

cRAG — pure C11, single binary, $0 dependencies. All figures reproducible: `make eval DATASET=scifact|nfcorpus`, `make check`, `bash tests/manual/answers.sh`.